Activity A: Predict and explain

1. Integer.parseInt("ABC");

It throws NumberFormatException, which is unchecked. I can prevent it by catching the exception or checking the text is a number first.

2. String name = null; name.length();

It throws NullPointerException, which is unchecked. I can prevent it by checking if (name != null) first.

3. new java.io.FileReader("missing.txt");

It throws FileNotFoundException, which is checked. I can handle it with try/catch or throws, or check that the file exists first.

OutOfMemoryError: It means the JVM has run out of memory, so the program is already in a bad state and will likely fail again if we continue. It is better to fix the cause, such as a memory leak or too much data.

Activity D: Engineer's review

1. IOException is the checked exception, and throws appears on my method: `static void importMarks(String filename) throws IOException`. File reading can fail for reasons outside my control, so Java forces me to handle or declare it. DuplicateStudentException is unchecked because it extends RuntimeException.

2. throw raises an exception, for example `throw new DuplicateStudentException(id);`. throws warns that a method may pass an exception to its caller, for example `importMarks(String filename) throws IOException`.

3. Users do not understand stack traces, so a friendly message like "Marks file not found; check the file name" tells them what to do. The logger keeps the technical details for the developer, and it keeps internal information away from users.

4. An empty catch block hides the error, so nothing is shown or logged and the problem is hard to find later. Catching Exception first means FileNotFoundException can never be reached, and Java gives a compile error. Specific catches must come before general ones.

Decision: skip or stop on an invalid record

I chose to skip an invalid record and continue. Each line is processed in its own try/catch inside the loop, so a non-numeric mark, an out-of-range mark, a wrong number of fields or a duplicate ID is reported and logged, and the importer moves on to the next line. This way one bad record does not prevent valid records from being imported. A missing file is different: nothing can be read, so that error is passed up to main(), which stops the import with a friendly message.

marks.txt

```
ICT001, 78
ICT002, ABC
ICT001, 65
ICT003, 90
```

MarksImporter.java

```
import java.io.*;
import java.util.*;
import java.util.logging.*;

public class MarksImporter {

    private static final Logger LOGGER =
            Logger.getLogger(MarksImporter.class.getName());

    // Custom exception: extends RuntimeException, so it is unchecked
    static class DuplicateStudentException extends RuntimeException {
        public DuplicateStudentException(String studentId) {
            super("Duplicate student ID: " + studentId);
        }
    }

    static void importMarks(String filename) throws IOException {
        Set<String> seen = new HashSet<>();

        // try-with-resources: the BufferedReader is declared in the
        // parentheses, so Java automatically calls br.close() when the
        // try block ends, whether it finishes normally or an exception occurs.
        try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
            String line;
            int lineNumber = 0;
            while ((line = br.readLine()) != null) {
                lineNumber++;
                try {
                    String[] parts = line.split(",");

                    // TODO 1: check there are exactly 2 fields
                    if (parts.length != 2) {
                        throw new IllegalArgumentException(
                                "Expected 2 fields but found " + parts.length);
                    }

                    String id = parts[0].trim();

                    // TODO 2: parse mark and validate range 0..100
                    int mark = Integer.parseInt(parts[1].trim());
                    if (mark < 0 || mark > 100) {
                        throw new IllegalArgumentException(
                                "Mark out of range (0-100): " + mark);
                    }

                    // TODO 3: detect repeated student ID using seen
```

```
                    if (!seen.add(id)) {
                        throw new DuplicateStudentException(id);
                    }

                    // TODO 4: display accepted record
                    System.out.println("Accepted: " + id + " -> " + mark);

                } catch (NumberFormatException e) {
                    System.out.println("Line " + lineNumber
                            + ": mark is not a number. Record skipped.");
                    LOGGER.log(Level.WARNING, "NumberFormatException on line "
                            + lineNumber + ": " + e.getMessage());
                } catch (DuplicateStudentException e) {
                    System.out.println("Line " + lineNumber
                            + ": duplicate student ID. Record skipped.");
                    LOGGER.log(Level.WARNING, "DuplicateStudentException on line
"
                            + lineNumber + ": " + e.getMessage());
                } catch (IllegalArgumentException e) {
                    System.out.println("Line " + lineNumber
                            + ": invalid record. Record skipped.");
                    LOGGER.log(Level.WARNING, "IllegalArgumentException on line "
                            + lineNumber + ": " + e.getMessage());
                }
            }
        }
    }

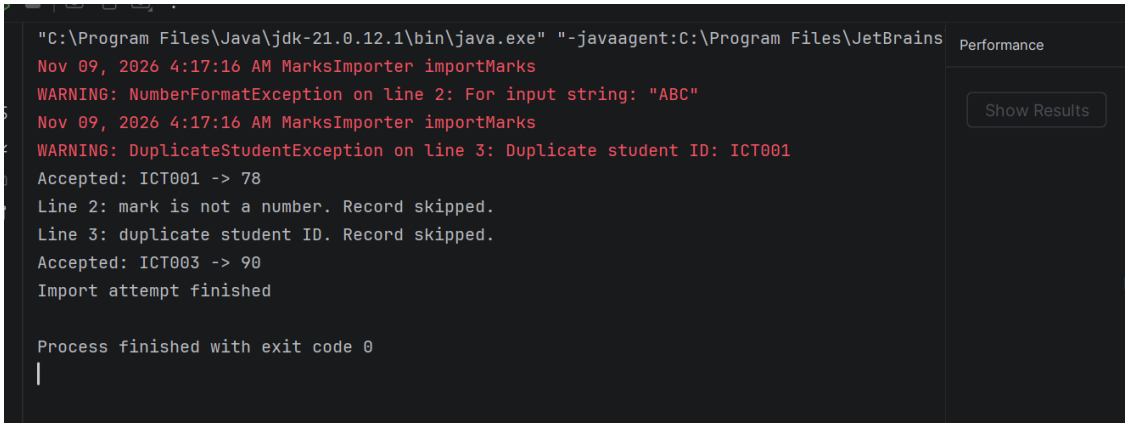
    public static void main(String[] args) {
        try {
            importMarks("marks.txt");
        } catch (FileNotFoundException e) {
            // Must come BEFORE IOException (it is the more specific type)
            System.out.println("Marks file not found; check the file name.");
            LOGGER.log(Level.SEVERE, "FileNotFoundException: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Could not read the marks file. Please try
again.");
            LOGGER.log(Level.SEVERE, "IOException: " + e.getMessage());
        } catch (DuplicateStudentException e) {
            System.out.println("A duplicate student was found.");
            LOGGER.log(Level.WARNING, "DuplicateStudentException: " +
e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid data was found in the file.");
            LOGGER.log(Level.WARNING, "IllegalArgumentException: " +
e.getMessage());
        } finally {
            System.out.println("Import attempt finished");
        }
    }
}
```

Test results (T1-T5)

T1: Valid mark accepted



T2: Invalid numeric value reported safely (line 2, ICT002,ABC)



T3: Duplicate student ID identified



T4: Out-of-range mark rejected



T5: Friendly file-not-found message; finally runs

